

Taller de Ecuaciones Diferenciales

Campos de pendientes y modelo de poblacion — codigo y graficas

Punto 1 — Campos de pendientes

Para cada ecuacion se traza el campo de pendientes y una familia de curvas solucion, resaltando en rojo la solucion particular que pasa por la condicion inicial. La integracion se hace en dos tramos desde x_0 (hacia atras hasta $x_{\min}$ y hacia adelante hasta $x_{\max}$) para garantizar que la curva pase exactamente por el punto dado.

Codigo

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

def campo(ax, f, xmin, xmax, ymin, ymax, n=20, largo=0.035):
    X, Y = np.meshgrid(np.linspace(xmin, xmax, n), np.linspace(ymin, ymax, n))
    with np.errstate(over="ignore", invalid="ignore"):
        M = f(X, Y)
    M = np.clip(np.nan_to_num(M, nan=0.0, posinf=1e6, neginf=-1e6), -1e6, 1e6)
    dx, dy = xmax - xmin, ymax - ymin
    esc = largo / np.sqrt((1/dx)**2 + (M/dy)**2)
    ax.quiver(X, Y, esc*M, angles="xy", scale_units="xy", scale=1,
              width=0.0035, color="0.45")

def resolver(f, x0, y0, xmin, xmax):
    tramos = []
    for destino in (xmin, xmax):
        if np.isclose(destino, x0):
            tramos.append((np.array([x0]), np.array([y0])))
            continue
        malla = np.linspace(x0, destino, 500)
        s = solve_ivp(f, [x0, destino], [y0], t_eval=malla, rtol=1e-8, atol=1e-10)
        tramos.append((s.t, s.y[0]))
    t = np.concatenate([tramos[0][0][:-1], tramos[1][0]])
    v = np.concatenate([tramos[0][1][:-1], tramos[1][1]])
    return t, v

def panel(f, nombre, ecu, xmin, xmax, ymin, ymax, x0, y0, archivo):
    fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

    for ax in axes:
        campo(ax, f, xmin, xmax, ymin, ymax)
        ax.set_xlim(xmin, xmax); ax.set_ylim(ymin, ymax)
        ax.set_xlabel("x"); ax.set_ylabel("y"); ax.grid(alpha=0.25)

    axes[0].set_title(f"{nombre} Campo de pendientes\n{ecu}")

    for v0 in np.linspace(ymin + 0.5, ymax - 0.5, 9):
        t, v = resolver(f, x0, v0, xmin, xmax)
        axes[1].plot(t, v, lw=1.2, color="tab:blue", alpha=0.65)

    t, v = resolver(f, x0, y0, xmin, xmax)
    axes[1].plot(t, v, lw=3, color="crimson", label="Solución particular")
    axes[1].scatter([x0], [y0], s=80, color="black", zorder=5,
                    label=f"y({x0:g}) = {y0:g}")
    axes[1].set_title(f"{nombre} Familia de soluciones")
    axes[1].legend(loc="best", fontsize=9)

    fig.tight_layout()
    fig.savefig(archivo, dpi=130)
    plt.close(fig)
```

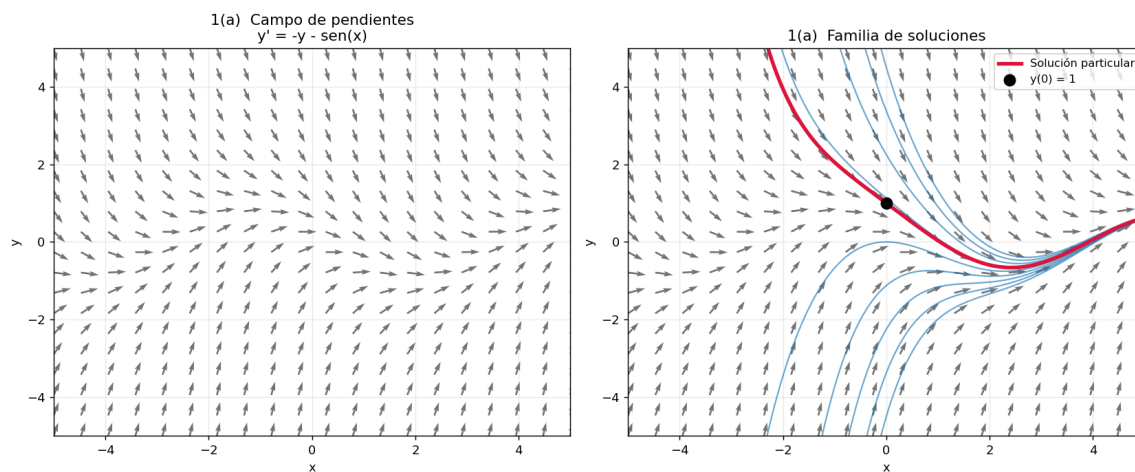
```

print("ok", archivo, "-> y(x0) =", np.interp(x0, t, v))

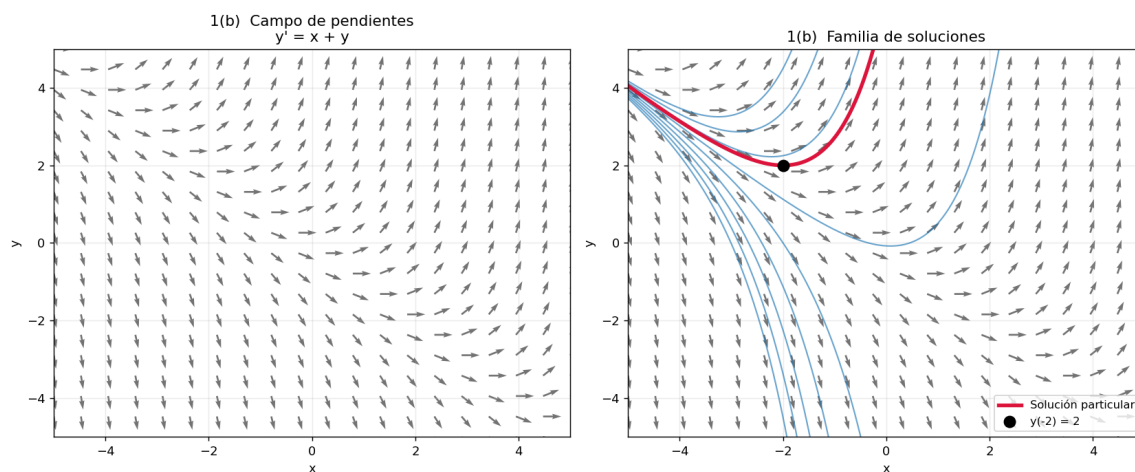
panel(lambda x,y: -y-np.sin(x), "1(a)", "y' = -y - sen(x)", -5,5,-5,5, 0,1, "1a.png")
panel(lambda x,y: x+y, "1(b)", "y' = x + y", -5,5,-5,5, -2,2, "1b.png")
panel(lambda x,y: -x**2+np.sin(y), "1(c)", "y' = -x^2 + sen(y)", -5,5,-10,5, 0,0, "1c.png")
panel(lambda x,y: (6*x-3*x*y)/(x**2+1), "1(d)", "(x^2+1)y' + 3xy = 6x", -5,5,-5,5, 0,3, "1d.png")
panel(lambda x,y: x*np.exp(np.clip(y,-50,5)), "1(e)", "y' = x*e^y", -1,1,-5,5, 0,0, "1e.png")
panel(lambda x,y: x-y, "1(f)", "y' = x - y", -5,5,-5,5, 1,1, "1f.png")

```

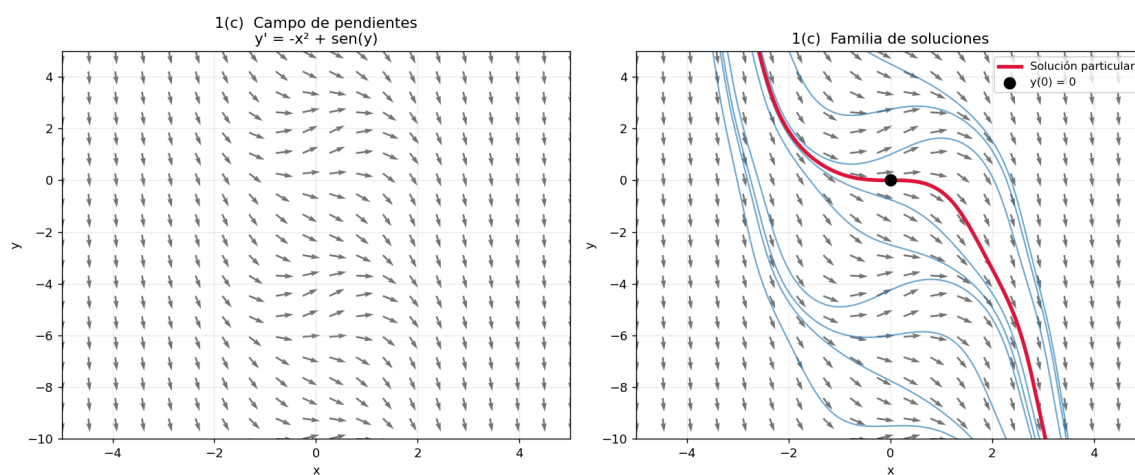
Graficas del punto 1



1(a) $y' = -y - \sin(x)$, $y(0) = 1$. Todas las soluciones convergen a la misma curva oscilatoria.

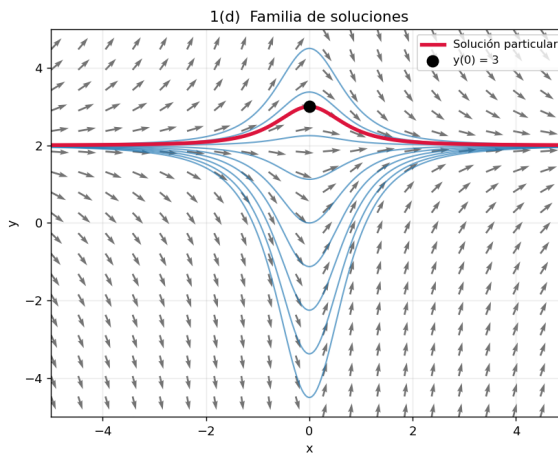
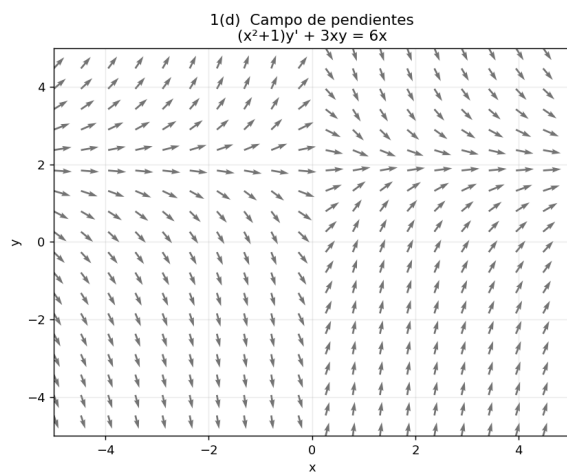


1(b) $y' = x + y$, $y(-2) = 2$. Divergencia exponencial a ambos lados de la recta $y = -x - 1$.

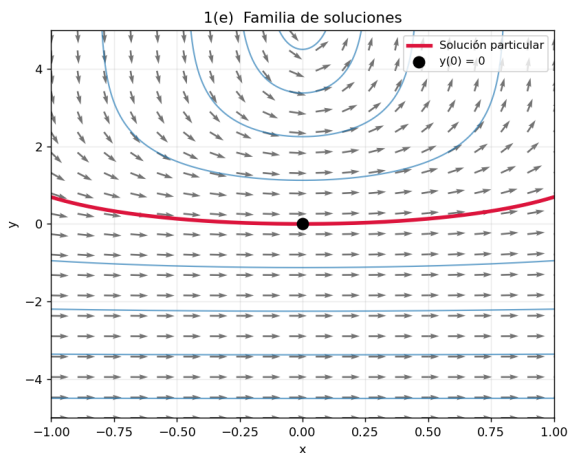
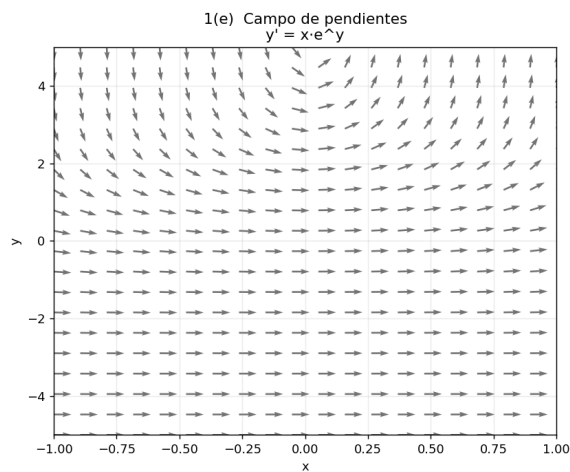


1(c) $y' = -x^2 + \sin(y)$, $y(0) = 0$. El término $-x^2$ domina y arrastra las soluciones hacia abajo.

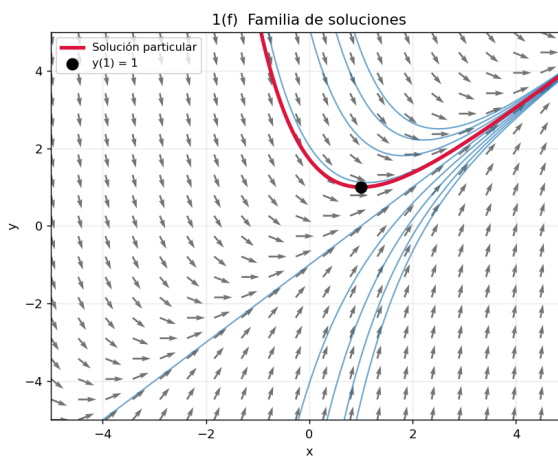
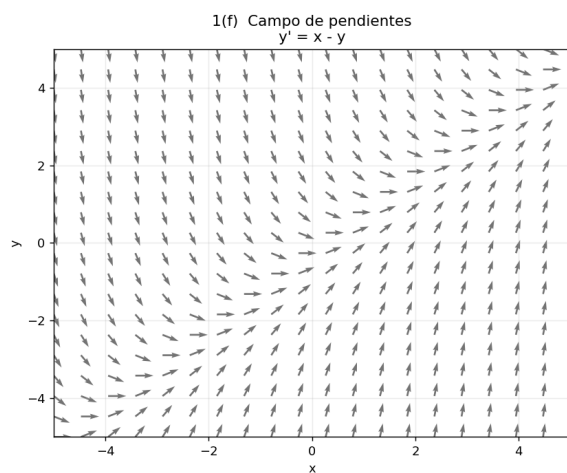
Graficas del punto 1 (continuacion)



1(d) $(x^2+1)y' + 3xy = 6x$, $y(0) = 3$. La recta $y = 2$ es solución de equilibrio y atrae a las demás.



1(e) $y' = x \cdot e^y$, $y(0) = 0$. Las soluciones que arrancan alto explotan en tiempo finito.



1(f) $y' = x - y$, $y(1) = 1$. Todas las soluciones se acercan a la asíntota $y = x - 1$.